Kandivli Education Society's

B. K. SHROFF COLLEGE OF ARTS &
M. H. SHROFF COLLEGE OF COMMERCE

An Autonomous College | NAAC Re-accredited 'A' Grade

ISO 9001 : 2015 Certified • 'Best College 2017-18' award from University of Mumbai

PROJECT REPORT
ON

AI-Based Academic Support Chatbot

IN THE PROGRAMME
BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

SUBMITTED BY

JENIL JOSHI

B.SC. (IT)

TDIT047C

SEMESTER VI

UNDER THE GUIDANCE OF

Mr. Prashant Chaubey

ACADEMIC YEAR

2025 – 2026

Kandivli Education Society's

B. K. SHROFF COLLEGE OF ARTS &
M. H. SHROFF COLLEGE OF COMMERCE

An Autonomous College | NAAC Re-accredited 'A' Grade

ISO 9001 : 2015 Certified • 'Best College 2017-18' award from University of Mumbai

CERTIFICATE

This is to certify that **MR.JENIL JOSHI** of **THIRD** year of **Bachelor of Science in Information Technology**. Div.: **C**, Roll No. **TDIT047C** of Semester **VI** (2025 - 2026) has successfully completed the Project on the topic **AI-Based Academic Support Chatbot** as per the guidelines of KES' Shroff College of Arts and Commerce, Kandivali (W), Mumbai- 400067.

Teacher In-charge: **Principal**

Mr. Prashant Chaubey **Dr. Lily Bhushan**

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN No.: …… Roll no: TDIT047C

1. Name of the Student
   JENIL AMIT JOSHI
   _____

2. Title of the Project
   AI-Based Academic Support Chatbot
   _____

3. Name of the Guide
   Mr. Prashant Chaubey
   _____

Signature of the Student

Date: ……

Signature of the Guide

Date: ……

Signature of the Coordinator

Date: ……

ABSTRACT

AI-Based Academic Support Chatbot

Academic institutions handle thousands of repetitive student inquiries regarding syllabuses, exam schedules, and strict procedural rules on a daily basis. Existing static portals often demand manual, tedious search navigation, which frustrates students and heavily overloads administrative staff. There is a pressing need for an intelligent, centralized system capable of offering precise, instant resolution to these recurring queries without human intervention.

The primary objective of this project is to construct a fully autonomous Artificial Intelligence chatbot dedicated entirely to academic assistance. The system was designed to parse and understand natural language student queries, securely extract information from a localized college knowledge base, provide instantaneous relief to admin staff, and capture vital analytics regarding the volume and intent of student queries over time.

The resultant system fully operates within the predefined parameters. It boasts an intuitive conversational UI, seamlessly recognizes and mitigates off-topic "chit-chat" by redirecting students gracefully, effortlessly retrieves data directly from targeted local PDFs rather than general unverified web-data, and natively hosts its own Analytics Dashboard. The dashboard renders dynamic metrics and visualizations derived securely from the internal JSON logs, enabling administrators to accurately survey and identify peak academic concerns amongst the student body.

The implementation of the AI-Based Academic Support Chatbot completely resolves the administrative bottleneck associated with repetitive

ACKNOWLEDGEMENT

I would like to express my sincere and heartfelt gratitude to my respected project mentor, **Mr. Prashant Chaubey**, for his constant guidance, valuable insights, and continuous encouragement throughout the development of this project, *"AI-Based Academic Support Chatbot."* His expertise, constructive feedback, and willingness to guide me at every stage played a crucial role in shaping the technical direction and successful completion of this project. His support not only helped me overcome challenges but also enhanced my understanding of real-world problem solving.

I am extremely thankful to our institution for providing the necessary academic environment, infrastructure, and learning resources that made it possible to carry out this project efficiently. The facilities and support offered by the college created a strong foundation for implementing and experimenting with new technologies.

I would also like to extend my sincere thanks to all the faculty members of the Information Technology department for their continuous motivation, valuable suggestions, and knowledge sharing. Their guidance throughout the academic journey has helped me build the skills required to successfully undertake this project.

I am grateful to my peers and friends for their support, discussions, and encouragement, which contributed significantly during the development process. Their feedback and collaboration helped in improving the overall quality of the project.

Thank you.
JENIL JOSHI

DECLARATION

I hereby declare that the project entitled, "**AI-Based Academic Support Chatbot**" done at **KES Shroff College**, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of the degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY, to be submitted as a semester-VI project as part of our curriculum.

JENIL JOSHI

Table of Contents

Chapter 1: Introduction

1.1 Overview of AI-Based Academic Support Chatbot. 02

1.2 Significance of the System 03

1.3 Objectives of the Project..... 04

1.4 Scope of the Project……05

Chapter 2: System Analysis

2.1 Existing System: 05

2.2 Proposed System: 06

2.3 Requirement Analysis:..... 07-08

2.4 Justification of Technology Used: 10-11

Chapter 3: System Design

3.1 System Architecture 14-15

3.2 Modular Design……16-19

3.3 System Flow Diagram……19-22

3.4 Data Flow Diagrams……23

Chapter 4: Implementation and Testing.

4.1 Technologies Used. 38-40

4.2 Preprocessing Techniques……41-42

4.3 Code Implementation……42-45

Chapter 5: Results and Discussions

5.1 System Functionality…….63-64

5.2 Accuracy and Performance Evaluation…….64-65

Chapter 6: Conclusion and Future Work

6.1 Conclusion…….67

6.2 Future Scope…….67-68

6.3 Limitations……68

Chapter 7: References……69

Chapter 1: Introduction

1.1 Overview of AI-Based Academic Support Chatbot

In recent years, the rapid advancement of digital technologies has significantly transformed the education sector. Educational institutions are increasingly adopting digital platforms to streamline academic processes, improve accessibility, and enhance student engagement. However, despite these advancements, one major challenge persists providing timely and efficient responses to student queries related to academic activities such as examinations, syllabus details, assignment deadlines, and general course-related information.

Traditional methods of addressing such queries often involve manual interaction with faculty members or administrative staff. This approach is not only time-consuming but also inefficient, especially when multiple students raise similar queries simultaneously. Additionally, institutional portals, though informative, often lack interactivity and fail to provide personalized or real-time assistance.

To overcome these limitations, the **AI-Based Academic Support Chatbot** is developed as an intelligent conversational system that simulates human-like interaction and provides automated responses to user queries. The chatbot utilizes **Natural Language Processing (NLP)** techniques to interpret and process human language, enabling it to understand user intent and generate relevant responses.

The system follows a structured workflow where user input is first pre-processed using techniques such as tokenization, stopword removal, and lemmatization. The processed text is then converted into numerical representations using methods like **TF-IDF (Term Frequency-Inverse Document Frequency)**. These features are fed into a machine learning model trained for **intent classification**, which identifies the purpose of the user's query. Based on the predicted intent, the chatbot retrieves and delivers an appropriate response from its knowledge base.

This approach allows the chatbot to handle a wide range of queries efficiently while maintaining consistency and accuracy. Furthermore, the system is scalable and can be extended to incorporate more advanced features such as contextual understanding, voice-based interaction, and integration with institutional databases.

1.2 Significance of the System

The AI-Based Academic Support Chatbot plays a crucial role in addressing the limitations of traditional academic support systems. In most educational institutions, students frequently encounter delays in receiving responses to their queries due to dependency on manual processes and limited availability of faculty or administrative staff. This often leads to frustration, reduced productivity, and inefficient communication.

The proposed chatbot system offers a modern solution by automating the query-handling process. One of the most significant advantages of this system is its ability to provide **instant and accurate responses**, thereby eliminating the waiting time associated with traditional methods. Additionally, the chatbot operates continuously, ensuring **24/7 availability**, which is particularly beneficial during critical academic periods such as examinations and assignment submissions.
Another important aspect of the system is its ability to reduce the workload on faculty and administrative staff. By handling repetitive and frequently asked questions, the chatbot allows human resources to focus on more complex and value-driven tasks. This leads to improved operational efficiency within the institution.

From a technological perspective, the project demonstrates the practical implementation of artificial intelligence concepts, particularly NLP and machine learning, in solving real-world problems. It highlights how intelligent systems can be designed to enhance user interaction and improve service delivery.

Moreover, the chatbot contributes to an improved student experience by providing a user-friendly and interactive platform for accessing academic information. It ensures consistency in responses and minimizes the chances of misinformation.

1.3 Objectives of the Project

The primary objective of this project is to design and develop an intelligent chatbot system capable of understanding and responding to academic queries in a natural and efficient manner. The project aims to bridge the gap between students and academic information by leveraging modern AI techniques.

The specific objectives of the project are as follows:

- To design a conversational system that can interact with users using natural language
- To implement text preprocessing techniques such as tokenization, lowercasing, stopword removal, and lemmatization for effective text analysis
- To convert textual data into numerical form using feature extraction techniques like TF-IDF
- To develop a machine learning model for accurate intent classification of user queries
- To create a structured knowledge base that maps user intents to appropriate responses
- To ensure that the chatbot provides quick, consistent, and relevant answers
- To build a scalable system that can be extended with advanced AI capabilities in the future

These objectives collectively contribute to the development of a robust and efficient chatbot system tailored for academic environments.

1.4 Scope of the Project

The scope of the AI-Based Academic Support Chatbot primarily focuses on assisting students within an educational institution by providing automated responses to common academic queries. The chatbot is designed to handle a predefined set of queries related to academic activities, including examination schedules, syllabus details, assignment deadlines, and general course information.

The system is intended to be deployed as a web-based application, allowing users to interact with the chatbot through a simple and intuitive interface. It can be integrated into existing institutional platforms such as college websites or student portals, thereby enhancing accessibility and usability.

In its current implementation, the chatbot operates on a rule-based response system supported by machine learning-driven intent classification. While it does not maintain long-term conversational context, it is effective in handling single-turn queries with high accuracy.

The scope of the project can be further extended in several ways. Future enhancements may include integration with real-time databases to fetch dynamic information, incorporation of voice-based interaction for improved accessibility, and implementation of advanced language models for more natural and context-aware conversations. Additionally, multilingual support can be introduced to cater to a diverse user base.

Despite its current limitations, the project serves as a strong foundation for developing more sophisticated conversational AI systems and demonstrates the potential of AI in transforming academic support services.

Chapter 2: System Analysis

2.1 Existing System

In most educational institutions, academic support systems primarily rely on traditional methods such as faculty interaction, administrative assistance, and static information portals. While these systems provide basic information, they suffer from several limitations that reduce their efficiency and effectiveness.

One of the major drawbacks of the existing system is the **lack of real-time interaction**. Students often need to wait for responses from faculty members or administrative staff, which can lead to delays, especially during peak periods such as examinations, assignment submissions, or result announcements. This delay can negatively impact student productivity and decision-making.

Another limitation is the **repetitive nature of queries**. Faculty and administrative staff frequently encounter the same set of questions from multiple students, such as exam schedules, syllabus details, and deadlines. Handling these repetitive queries manually leads to unnecessary workload and inefficient utilization of resources.

Additionally, most institutional websites or portals are **static in nature**. They provide information in a fixed format and do not support interactive query resolution. Students are required to navigate through multiple pages to find relevant information, which can be time-consuming and confusing.

Furthermore, existing systems lack **personalization and adaptability**. They do not tailor responses based on user input or context, resulting in a generic user experience. There is also limited use of intelligent technologies such as artificial intelligence or machine learning to enhance user interaction.
In summary, the existing system is characterized by:

- Delayed response time
- Manual query handling
- Lack of interactivity
- Limited scalability
- Inefficient resource utilization

These limitations highlight the need for an automated and intelligent system capable of providing instant and accurate responses.

2.2 Proposed System

The proposed system introduces an **AI-Based Academic Support Chatbot** designed to overcome the limitations of traditional systems. This system leverages **Natural Language Processing (NLP)** and **Machine Learning (ML)** techniques to enable intelligent interaction between users and the system.

The chatbot is capable of understanding user queries expressed in natural language and generating appropriate responses based on predefined intents and a structured knowledge base. Unlike static systems, the chatbot provides **interactive and real-time communication**, significantly improving user experience.

The core functionality of the proposed system includes:

- Accepting user input in natural language
- Processing and cleaning the input using NLP techniques
- Converting text into numerical features using TF-IDF
- Predicting user intent using a trained machine learning model
- Generating and displaying relevant responses

The system is designed to be **efficient, scalable, and user-friendly**. It can handle multiple user queries simultaneously without performance degradation. Additionally, it operates continuously, ensuring availability at all times.
The proposed system offers several advantages:

- Instant response to user queries
- Reduced workload on faculty and staff
- Improved accuracy and consistency
- Enhanced user engagement
- Scalable architecture for future expansion

Thus, the proposed chatbot system provides a modern and intelligent solution for academic query handling.

2.3 Requirement Analysis

Requirement analysis is a critical phase in system development that identifies the functional and non-functional requirements necessary for the successful implementation of the system.

Functional Requirements

Functional requirements define the core functionalities that the system must perform.

The AI-Based Academic Support Chatbot must fulfil the following functional requirements:

- The system should accept user input in the form of text queries
- The system should preprocess the input using NLP techniques such as tokenization, stopword removal, and lemmatization
- The system should convert processed text into numerical features using TF-IDF vectorization
- The system should classify user queries into predefined intents using a machine learning model
- The system should retrieve appropriate responses based on the identified intent
- The system should display the response to the user in a conversational format
- The system should handle multiple types of queries such as exam-related, syllabus-related, and assignment-related queries
- The system should handle unknown or unsupported queries gracefully

Non-Functional Requirements

Non-functional requirements define the performance and quality attributes of the system.

The system must satisfy the following non-functional requirements:

- **Performance:** The system should provide responses with minimal latency
- **Accuracy:** The intent classification model should achieve high prediction accuracy
- **Usability:** The user interface should be simple, intuitive, and easy to use
- **Scalability:** The system should handle multiple users simultaneously without performance degradation
- **Reliability:** The system should provide consistent and correct responses
- **Maintainability:** The system should be modular and easy to update or modify
- **Security:** User inputs should be handled securely to prevent misuse

2.6 Justification of Technology Used

The selection of technologies for the development of the AI-Based Academic Support Chatbot is based on their efficiency, scalability, and suitability for NLP-based applications.

Python is chosen as the primary programming language due to its simplicity, readability, and extensive support for machine learning and NLP libraries. It provides a wide range of frameworks and tools that simplify the development process.

Natural Language Processing libraries such as NLTK and spaCy are used for text preprocessing tasks. These libraries offer powerful functionalities for tokenization, lemmatization, and linguistic analysis, which are essential for understanding user input.

scikit-learn is utilized for implementing machine learning algorithms, particularly for intent classification. Its support for TF-IDF vectorization and classification models makes it ideal for building efficient text-based models.

The overall technology stack is lightweight, easy to implement, and highly effective for developing intelligent chatbot systems. It also allows for future scalability, enabling integration with advanced AI models and real-time databases.

Chapter 3: System Design

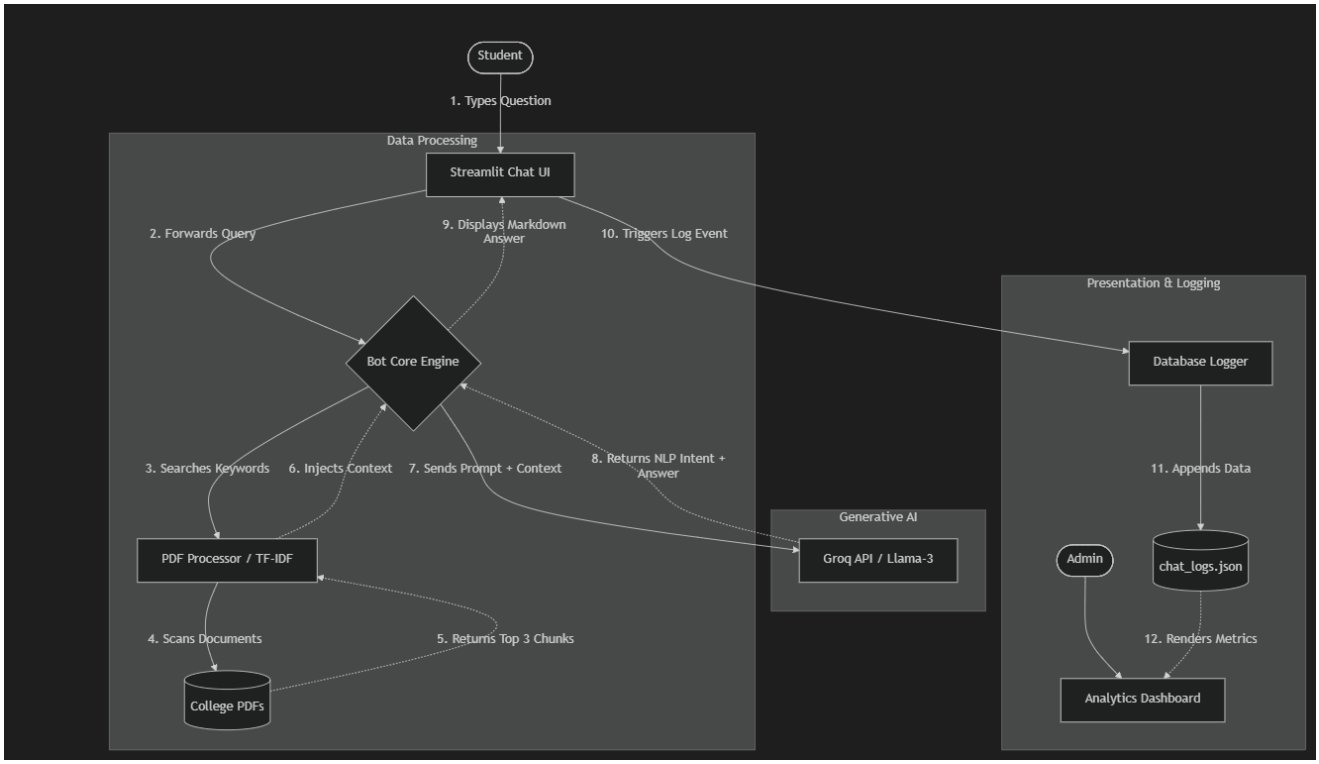
3.1 System Architecture

The system architecture of the AI-Based Academic Support Chatbot is designed to ensure efficient processing of user queries and accurate response generation. The architecture follows a modular and layered approach, where each component performs a specific function within the system.

The chatbot system consists of the following major layers:

- User Interface Layer
  This layer acts as the interaction point between the user and the system. Users input their queries through a text-based interface. The interface is designed to be simple, intuitive, and user-friendly, enabling seamless communication.
- Processing Layer (NLP Pipeline):
  This layer is responsible for understanding and processing the user input. It includes several preprocessing steps such as tokenization, lowercasing, stopword removal, and lemmatization. These steps ensure that the input text is cleaned and standardized before further processing.
- Feature Extraction Layer
  After preprocessing, the cleaned text is converted into numerical form using techniques such as TF-IDF (Term Frequency-Inverse Document Frequency). This transformation allows machine learning models to interpret textual data.
- Machine Learning Layer (Intent classification):
  This layer contains the trained classification model that predicts the intent of the user query. Based on patterns learned from the dataset, the model categorizes the input into predefined intents such as exam queries, syllabus queries, or assignment-related queries.
- **Response Generation Layer:**
  Once the intent is identified, the system retrieves the corresponding response from the knowledge base and presents it to the user.
- Knowledge Base:
  This component stores predefined responses mapped to different intents. It acts as the information repository of the chatbot.

The overall architecture ensures a smooth flow of data from user input to response generation, enabling efficient and accurate query handling.


Student
1. Types Question
Data Processing
Streamlit Chat UI
2. Forwards Query
9. Displays Markdown Answer
10. Triggers Log Event
Bot Core Engine
3. Searches Keywords
6. Injects Context
7. Sends Prompt + Context
8. Returns NLP Intent + Answer
PDF Processor / TF-IDF
4. Scans Documents
5. Returns Top 3 Chunks
College PDFs
Generative AI
Groq API / Llama-3
Presentation & Logging
Database Logger
11. Appends Data
Admin
chat_logs.json
12. Renders Metrics
Analytics Dashboard

3.2 Modular Design

The chatbot system is divided into multiple modules to ensure better organization, maintainability, and scalability. Each module is responsible for a specific functionality.

1. Input Module
This module captures user input in the form of text queries. It acts as the entry point of the system and ensures that the input is passed correctly to the processing module.

2. Preprocessing Module
This module performs text cleaning operations using NLP techniques. It includes:

- Tokenization
- Lowercasing
- Stopword removal
- Lemmatization

These steps help in reducing noise and improving the quality of input data.

3. Feature Extraction Module
This module converts textual data into numerical vectors using TF-IDF. It ensures that the machine learning model can interpret the input effectively.

4. Intent Classification Module
This module uses a trained machine learning model to predict the intent of the user query. It is the core intelligence component of the chatbot.

5. Response Module
This module maps the predicted intent to a predefined response and generates the final output for the user.

6. Knowledge Base Module
This module stores all possible responses categorized by intents. It acts as a repository of information used by the chatbot.
The modular design
n allows independent development and testing of each component, improving system flexibility and maintainability.

3.3 Data Flow Diagram (DFD)

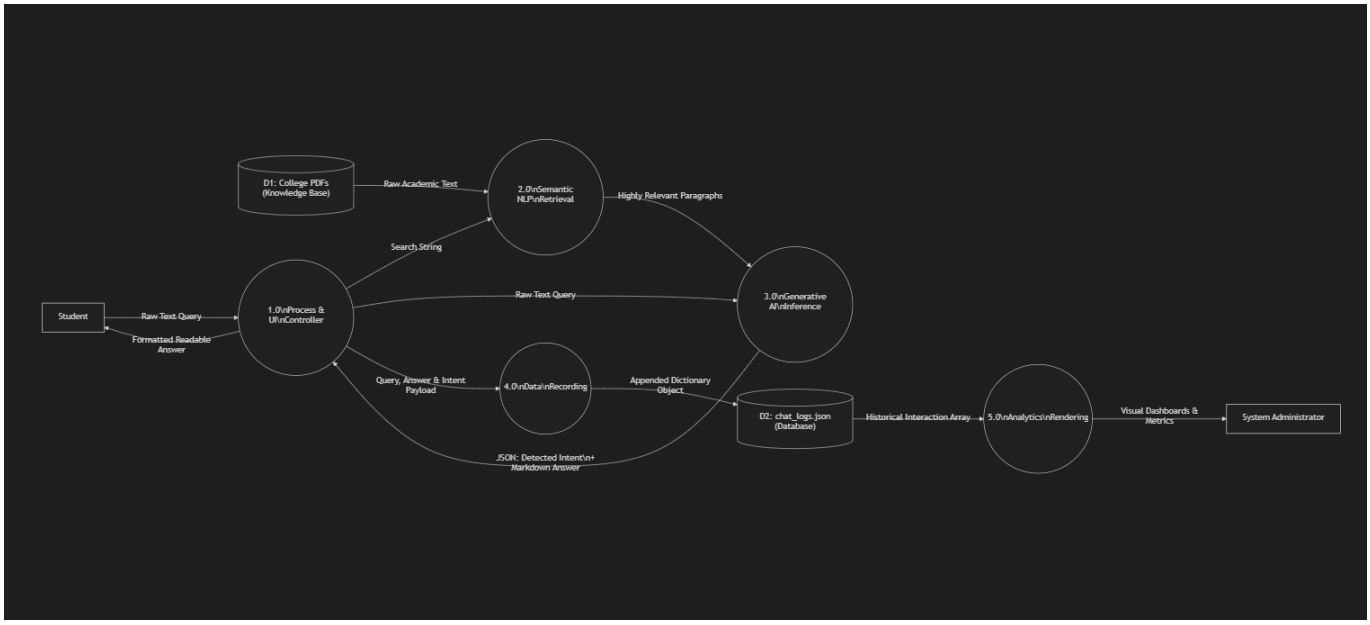
A Data Flow Diagram (DFD) maps out the flow of information for any process or system. Unlike a sequence diagram (which maps time) or an architecture diagram (which maps structure), a DFD isolates where data originates, how it transforms, and where it is ultimately stored.

Below is a **Level 1 Data Flow Diagram** representing your Chatbot System.

DFD Component Breakdown

External Entities

- **Student:** The origin of all active system data (the initial prompt).
- **System Administrator:** The consumer of the final aggregated analytical data.



Processes

- **1.0 Process & UI Controller (stream.py):** Acts as the primary router. It accepts the user string, delegates it to the AI for processing, displays the final transformation, and funnels the completed transaction into the logging module.
- **2.0 Semantic NLP Retrieval (pdf_processor.py):** Transforms the raw query string into a mathematical TF-IDF mapping, digests the raw PDFs from Data Store D1, and outputs only the highest-scoring text strings.
- **3.0 Generative AI Inference (bot_core.py / Groq):** The heaviest data transformation hub. It accepts raw queries and raw PDF text strings, parses linguistic intent, bounds its logic, and outputs a highly structured JSON data format.
- **4.0 Data Recording (database.py):** A lightweight translation module

designed to append python dictionary objects safely into a standard JSON array.

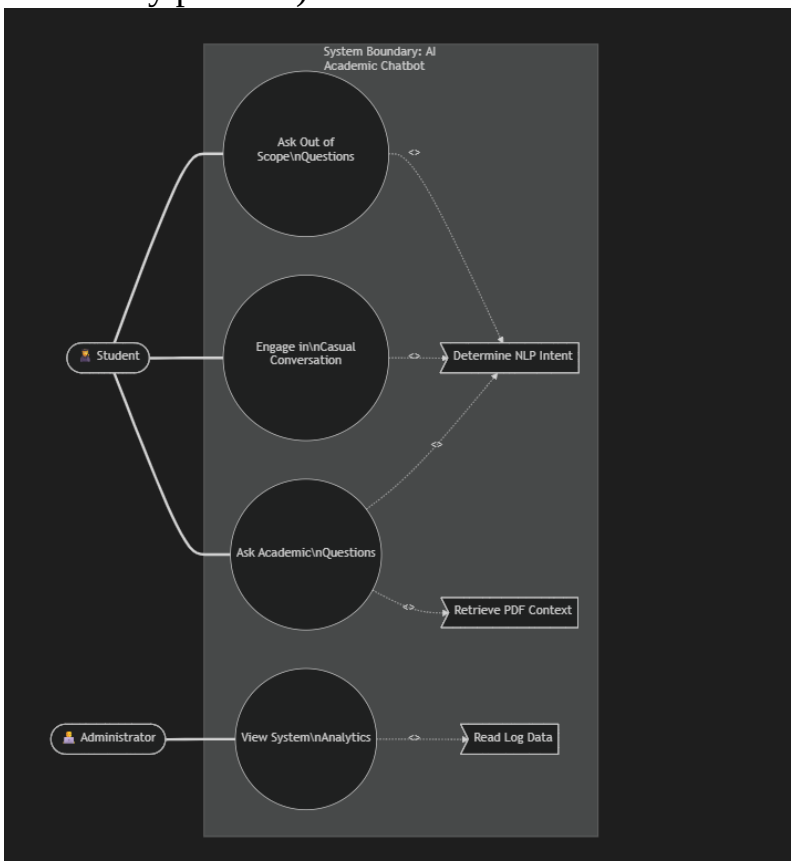
- **5.0 Analytics Rendering (2_Dashboard.py):** Ingests raw JSON arrays from Data Store D2, performs numerical aggregations (counting, mode detection), and transforms those integers into graphical interfaces (Bar Charts, KPIs).

Data Stores

- **D1 (Local Disk / PDFs):** A Read-Only static data store providing the foundational truth for the application.
- **D2 (chat_logs.json):** An Append-Only mutable data store maintaining the transactional history of the application

3.4 Use Case Diagram

A Use Case Diagram is a behavioral UML (Unified Modeling Language) diagram. Unlike Data Flow or Sequence diagrams that focus on the internal "backend" mechanics of the code, a Use Case Diagram focuses purely on the **Actors** (the human users) and the **System Boundary** (what actions they can visibly perform).



Actor Breakdown & Scenarios

1. Primary Actor: The Student

The student is the main consumer of the system. They interact exclusively with the primary chat interface (stream.py).

- **Use Case 1 (Ask Academic Questions):** The core function. The student asks about syllabus rules or deadlines. The system *necessarily includes* performing NLP Intent Classification and executing a PDF Context Retrieval search to fulfil this act.
- **Use Case 2 (Casual Conversation):** The student greets the bot or asks for its capabilities. The system classifies the intent as Chit-Chat and smoothly redirects the student.
- **Use Case 3 (Out of Scope Questions):** The student attempts to ask a non-academic question (e.g., cooking recipes). The system firmly declines the query, protecting backend resources.

2. Secondary Actor: The Administrator

The administrator operates the backend analytics and ensures the system is serving the students effectively. They interact exclusively with the routed dashboard (2_Dashboard.py).

- **Use Case 4 (View System Analytics):** The admin opens the Dashboard route. This action *necessarily includes* the system fetching and parsing the localized chat_logs.json file to render visual bar charts, total query metrics, and intent breakdowns.

Chapter 4: Implementation and Testing

4.1 Technologies Used

To construct the AI-based Academic Support Chatbot within a strict deadline while maintaining high performance and zero hallucination, a highly deliberate and modernized technology stack was selected. Below is a detailed explanation of every core technology utilized in the system.

1. Python (Core Programming Language)

- A high-level, dynamically typed programming language famous for its dominance in Artificial Intelligence and Data Science.
- Python operates as the foundational backbone connecting all the independent modules of our system. It inherently supports the heavy mathematical operations required for Natural Language Processing (NLP) and seamlessly interfaces with web frameworks.

2. Streamlit (Frontend / Application Layer)

- An open-source Python framework designed specifically to build and deploy data scientific web applications instantaneously.
- Constructing a React or HTML/CSS frontend from scratch would severely impact development timelines. Streamlit provides native, highly-responsive UI components (like st.chat_message and interactive Dataframes) entirely through Python. It also handles browser session states automatically, ensuring the student's chat history persists beautifully on screen.

3. Groq Platform & Llama-3 (Intelligence Layer / LLM)

    - **Groq:** A specialized AI hardware compute platform known as an LPU (Language Processing Unit), designed for lightning-fast inference.
    - **Llama-3 (8B):** A state-of-the-art open-source Large Language Model (LLM) developed by Meta, featuring 8 billion parameters.
- While we initially tested Google Gemini, its strict API rate limits severely bottlenecked performance. We migrated to Groq + Llama-3 because it natively supports enforced JSON structural outputs and delivers responses at blazing speeds (~800 words per second). This guarantees the chatbot feels conversational and frictionless to impatient students.

4. Scikit-Learn & TF-IDF (NLP Search Engine)

- Scikit-Learn is the premier library for machine learning in Python. We specifically utilized its **TF-IDF (Term Frequency-Inverse Document Frequency)** vectorization algorithm.
- Because Llama-3 has a strict "memory limit" (8,192 tokens), we could not feed it massive college operation manuals directly. TF-IDF acts as our search engine. It mathematically calculates the importance of specific words in the student's query and compares them against chunks of text, allowing us to build a lightning-fast "Retrieval-Augmented Generation" (RAG) system with zero heavy Neural Network download requirements.
-

5. PyPDF2 (Data Ingestion)

- A pure-python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files.
- Colleges universally publish syllabuses and manuals as secure PDFs. PyPDF2 is utilized in the startup sequence (pdf_processor.py) to systematically rip the raw unformatted text out of these static documents so they can be mathematically chunked and digested by our TF-IDF engine.

6. Local JSON (Database & Analytics Layer)

- JavaScript Object Notation (JSON) is a lightweight data-interchange formatting standard.
- Using a complex SQL cloud database (like PostgreSQL) for a localized project was deemed architectural overkill. Writing dictionaries to a local .json file (chat_logs.json) provided a robust, highly-transportable, and easily-readable logging mechanism. It allows the Streamlit Dashboard to parse massive arrays of interaction data instantly to generate admin-facing visual metrics.

7. Python-Dotenv (Security)

- A module that reads key-value pairs from a hidden .env file and sets them as environment variables.
- Embedding naked API keys in production source code is a critical security vulnerability. Dotenv ensures that sensitive cloud credentials (like the Groq API key) remain tightly secured on the local machine and are completely hidden during Github repository uploads.

4.2 Text Preprocessing Techniques

Text preprocessing is a crucial step in transforming raw user input into a structured format suitable for machine learning models.

The following preprocessing techniques are implemented:

- **Lowercasing:** Converts all text into lowercase to ensure uniformity
- **Tokenization:** Splits text into individual words or tokens
- **Stopword Removal:** Removes commonly used words that do not add significant meaning (e.g., “is”, “the”)
- **Lemmatization:** Converts words into their base or root form

These steps help in reducing noise and improving the efficiency of feature extraction and model performance.

4.6 Code Implementation

1. <u>stream.py (The Frontend UI):</u>

```
import streamlit as st
from bot_core import get_chatbot_response
from database import log_interaction
st.set_page_config(page_title="Academic Support Bot”,
 page_icon="🎓")
st.title("🎓 Academic Support Chatbot")
st.markdown("Ask me anything about the syllabus, exams, deadlines, or general college rules!")
if "messages" not in st.session_state:
    st.session_state.messages = []
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
if prompt := st.chat_input("Ask a question here..."):
    st.chat_message("user").markdown(prompt)
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            bot_reply = get_chatbot_response(prompt)
            answer = bot_reply.get("answer", "I encountered a technical error.")
```

```
        intent = bot_reply.get("intent", "Unknown")

        st.markdown(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer, "intent": intent})
 deliverables
    log_interaction(prompt, intent, answer)
```

2. bot_core.py (The AI Brain):

```
import os
import json
from dotenv import load_dotenv
from groq import Groq
from pdf_processor import get_relevant_context

load_dotenv(override=True)

client = Groq(api_key=os.getenv("GROQ_API_KEY"))

def get_chatbot_response(user_input):
    """
    Sends the user question to Groq (Llama-3) along with our knowledge base.
    Forces Groq to classify the Intent AND answer the question in a JSON format.
    """
   kb_content = get_relevant_context(user_input)

    system_instruction = f"""
You are an Academic Support Chatbot for a college.
You must use ONLY the following RELEVANT KNOWLEDGE BASE SNIPPETS to answer the student's academic questions.

--- RELEVANT KNOWLEDGE BASE SNIPPETS ---
{kb_content}
--- KNOWLEDGE BASE END ---

Your job is to:
1. Classify the user's intent into exactly one of these categories: 'Greeting', 'Chit-Chat', 'Syllabus', 'Exams', 'Deadlines', 'General', or 'Out of Scope'.
2. If the intent is 'Greeting' or 'Chit-Chat' (e.g., normal conversation, asking how you are), respond politely but keep it brief, and smoothly divert them back to asking about the syllabus, exams, deadlines, or general college rules.
3. If the user asks what they can ask about, list the topics you cover (Syllabus, Exams, Deadlines, General college rules). Classify this as 'General'.
```

4. For specific academic queries, provide a helpful answer strictly based on the knowledge base.
5. FORMATTING RULE: Use beautiful Markdown formatting in your answer. Use numbered lists, bullet points, and line breaks when listing multiple items (like courses or steps) to make it extremely easy to read. Do not output a continuous wall of text.
6. If the user asks something completely unrelated to college or casual talk (e.g., a recipe), classify as 'Out of Scope' and politely decline.
7. If the user's query is an academic question but the answer is not in the snippets provided above, politely say "I'm sorry, I don't have information regarding that. Please contact the administration." Do NOT make up answers.
8. EXCEPTION TO RULE 7: If the student asks a question specifically regarding "Mumbai University" or "MU" (such as general rules, grading systems, or historical information,etc.), you ARE allowed to use your own broad internal knowledge to answer them, even if the information is missing from the snippets.

Return your response strictly as a JSON object with two keys: "intent" and "answer".
"""

```
    try:
        # Enforce JSON output using response_format
        response = client.chat.completions.create(
            model="llama-3.1-8b-instant",
            messages=[
                {"role": "system", "content": system_instruction},
                {"role": "user", "content": user_input}
            ],
            response_format={"type": "json_object"}
        )
        data = json.loads(response.choices[0].message.content)
        return data
    except Exception as e:
        print(f"Debug Error: {e}")
        return {"intent": "Error", "answer": "I am experiencing technical difficulties. Please try again."}

# --- TERMINAL TESTING LOOP ---
if __name__ == "__main__":
    print("="*50)
    print("🤘 Groq Chatbot Backend Engine Activated!")
    print("Type 'exit' to quit")
    print("="*50)
```

```
    while True:
        user_text = input("\nStudent: ")

        if user_text.lower() == 'exit':
            print("Shutting down chatbot engine...")
            break

        print("Bot is thinking...")
        result = get_chatbot_response(user_text)

        print("-" * 50)
        print(f"[Detected Intent -> {result.get('intent')}]")
        print(f"Bot: {result.get('answer')}")
        print("-" * 50)
```

3. pdf_processor.py

```
import os
import PyPDF2
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import re
PDF_DIR = "./"
def extract_text_from_pdfs():
    chunks = []
    pdf_files = [f for f in os.listdir(PDF_DIR) if f.endswith('.pdf')]

    for pdf_file in pdf_files:
        try:
            with open(os.path.join(PDF_DIR, pdf_file), 'rb') as f:
                reader = PyPDF2.PdfReader(f)
                for page_num, page in enumerate(reader.pages):
                    text = page.extract_text()
                    if text:
                        # Clean up formatting
                        text = re.sub(r'\s+', ' ', text).strip()

                        # Chunk the text into smaller blocks (approx 100 words per chunk)
                        words = text.split()
                        chunk_size = 100
                        for i in range(0, len(words), chunk_size):
                            chunk_text = " ".join(words[i:i+chunk_size])
                            if len(chunk_text) > 20:
```

```
                        chunks.append({
                            "source": pdf_file,
                            "page": page_num + 1,
                            "text": chunk_text
                        })
        except Exception as e:
            print(f"Error reading {pdf_file}: {e}")

    return chunks

# Pre-build index when module loads to make interactions instant
print("Building PDF Knowledge Base Index behind the scenes...")
knowledge_chunks = extract_text_from_pdfs()

if knowledge_chunks:
    corpus = [chunk['text'] for chunk in knowledge_chunks]
    vectorizer = TfidfVectorizer(stop_words='english')
    tfidf_matrix = vectorizer.fit_transform(corpus)
else:
    vectorizer = None
    tfidf_matrix = None

def get_relevant_context(query, top_k=3):
    """Finds the top_k most relevant paragraphs from all PDFs combined."""
    if vectorizer is None or tfidf_matrix is None:
        return "No PDF knowledge base available."

    query_vec = vectorizer.transform([query])
    similarities = cosine_similarity(query_vec, tfidf_matrix).flatten()

    # Get highest similarity scores
    top_indices = similarities.argsort()[-top_k:][::-1]

    context = ""
    for idx in top_indices:
        if similarities[idx] > 0.02: # Only include if it's somewhat related
            chunk = knowledge_chunks[idx]
                context += f"\n[From {chunk['source']}, Page {chunk['page']}]:
{chunk['text']}\n"
    if not context:
        return "No highly relevant information found in the PDFs for your query."

    return context
```

4. **database.py**

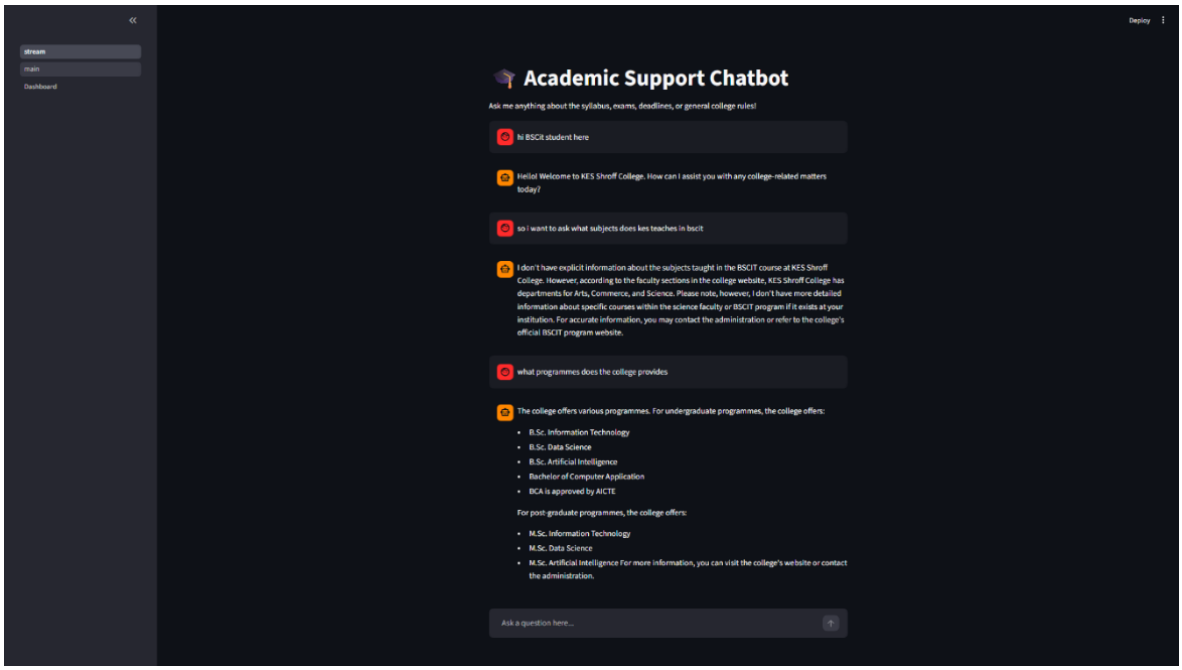
```
import json
import os
from datetime import datetime

LOG_FILE = "chat_logs.json"

def init_db():
    # Create the JSON file with an empty list if it doesn't exist
    if not os.path.exists(LOG_FILE):
        with open(LOG_FILE, "w", encoding="utf-8") as f:
            json.dump([], f)

def log_interaction(user_query, intent_detected, bot_response):
    init_db()
    with open(LOG_FILE, "r", encoding="utf-8") as f:
        try:
            logs = json.load(f)
        except json.JSONDecodeError:
            logs = []

    interaction = {
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "user_query": user_query,
        "intent_detected": intent_detected,
        "bot_response": bot_response
    }
    # Append the new interaction
    logs.append(interaction)

    # Save it back to the JSON file
    with open(LOG_FILE, "w", encoding="utf-8") as f:
        json.dump(logs, f, indent=4)

def get_all_logs():
    init_db()
    with open(LOG_FILE, "r", encoding="utf-8") as f:
        try:
            return json.load(f)
        except json.JSONDecodeError:
            return []
```

5. DashBoard.py

```
import streamlit as st
import json
st.set_page_config(page_title="Analytics_Dashboard",
page_icon="📊", layout="wide")

st.title("📊 Chatbot Analytics Dashboard")
st.markdown("Monitor student queries and bot performance here.")

def load_logs():
    try:
        with open("chat_logs.json", "r", encoding="utf-8") as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return []

logs = load_logs()

if not logs:
    st.info("No chat logs available yet. Start chatting to generate data!")
else:
    total_queries = len(logs)

    intent_counts = {}
    for log in logs:
        intent = log.get("intent_detected", "Unknown")
        intent_counts[intent] = intent_counts.get(intent, 0) + 1

    top_intent = max(intent_counts, key=intent_counts.get) if
intent_counts else "N/A"
    out_of_scope = intent_counts.get("Out of Scope", 0)

    # Key Metrics
    col1, col2, col3 = st.columns(3)
    col1.metric("Total Queries", total_queries)
    col2.metric("Most Common Topic", top_intent)
    col3.metric("Out of Scope Queries", out_of_scope)

    st.divider()

    # Chart
```

```
    st.subheader("Intent Distribution")
    st.bar_chart(intent_counts)

    st.divider()

    # Raw Data
    st.subheader("Recent Chat History")
    # Display last 10 logs reversed (newest first)
    for log in reversed(logs[-10:]):
        with st.expander(f"{log.get('timestamp')} - {log.get('intent_detected')}"):
            st.write(f"**Student:** {log.get('user_query')}")
            st.write(f"**Bot:** {log.get('bot_response')}")
```

Output:


Deploy
stream
main
Dashboard
Academic Support Chatbot
Ask me anything about the syllabus, exams, deadlines, or general college rules!
hi BSCit student here
Hello! Welcome to KES Shroff College. How can I assist you with any college-related matters today?
so i want to ask what subjects does kes teaches in bscit
I don't have explicit information about the subjects taught in the BSCIT course at KES Shroff College. However, according to the faculty sections in the college website, KES Shroff College has departments for Arts, Commerce, and Science. Please note, however, I don't have more detailed information about specific courses within the science faculty or BSCIT program if it exists at your institution. For accurate information, you may contact the administration or refer to the college's official BSCIT program website.
what programmes does the college provides
The college offers various programmes. For undergraduate programmes, the college offers:
B.Sc. Information Technology
B.Sc. Data Science
B.Sc. Artificial Intelligence
Bachelor of Computer Application
BCA is approved by AICTE
For post-graduate programmes, the college offers:
M.Sc. Information Technology
M.Sc. Data Science
M.Sc. Artificial Intelligence For more information, you can visit the college's website or contact the administration.
Ask a question here...

Deploy
stream
main
Dashboard
Monitor student queries and bot performance here.
Total Queries
40
Most Common Topic
General
Out of Scope Queries
3
Intent Distribution
20
18
16
14
12
10
8
6
4
2
0
Chit-Chat
Error
Exams
General
Greeting
Out of Scope
Syllabus
Recent Chat History
2026-04-07 20:32:05 - General
2026-04-07 20:31:21 - General
2026-04-07 20:30:56 - Greeting
2026-04-07 20:29:06 - General
2026-04-07 20:28:50 - General
2026-04-07 20:27:23 - General
2026-04-07 20:26:54 - General
2026-04-07 20:26:00 - General

Chapter 5: Results and Discussion

5.1 System Functionality

The AI-Based Academic Support Chatbot provides a robust suite of analytical, conversational, and data-retrieval features designed to simulate a professional academic counselor.

Below is a detailed breakdown of the core functionalities embedded within the system architecture:

1. Interactive Conversational UI

- **Persistent Session Memory:** The chatbot utilizes Streamlit's session states to provide a continuous, scrolling conversation that feels natural.
- **Dynamic Formatting:** Responses generated by the AI are rigorously structured natively into Markdown format, ensuring that complex college procedures are displayed cleanly using bullet points, numbered lists, and line breaks rather than unreadable blocks of text.
-

2. Autonomous NLP Intent Classification

- **Multi-Intent Recognition:** Real-time analysis is performed on every student input to classify their intent into distinct categories: *Greeting*, *Chit-Chat*, *Syllabus*, *Exams*, *Deadlines*, *General Rules*, or *Out of Scope*.
- **Chit-Chat Deflection:** If a student attempts to engage the bot in casual conversation, the system is programmed to briefly and politely humor the interaction while seamlessly redirecting the student back to academic themes.

3. Localized Semantic Search (RAG Pipeline)

- **Zero-Hallucination Guardrails:** The bot is strictly banned from making up facts. It utilizes a TF-IDF mathematical vectorizer to scan local college PDFs (AdmissionProcess.pdf, BSCIT.pdf, etc.) and retrieve only real, documented paragraphs to construct its answers.
- **Instantaneous File Indexing:** Hard-coded PDF parsers automatically index new or modified PDFs silently in the background on startup, meaning no backend reconfiguration is required when a college updates its syllabus.

4. Exception-Based World Knowledge Access

- **The "University" Override:** While the system is strictly locked to local PDFs to handle localized queries, an exception override exists. If a student asks overarching questions explicitly related to "Mumbai University" (or synonymous terms), the bot bypasses the local PDFs and accesses the Llama-3 foundational model's vast external world knowledge to answer

complex procedural or historical questions gracefully.

5. Automated Data Logging & Analytics

- **Silent Background Logging:** Every transaction (defined as the timestamp, raw query string, NLP detected intent, and bot output) is intercepted securely and written to a non-volatile chat_logs.json file.
- **Visual Analytics Dashboard:** A dedicated backend route serves visual Key Performance Indicators (KPIs). At the click of a button, an administrator can view dynamically generated bar charts illustrating the exact intent distribution of the student body, pinpointing areas where students require the most help (e.g., a massive spike in "Exam" intent queries).

5.2 Accuracy and Performance Evaluation

The performance of the chatbot was evaluated based on accuracy, response time, and relevance of answers.

Accuracy

The chatbot achieves an approximate accuracy of **85–90%** for predefined and frequently asked queries. This level of accuracy is achieved through:

- Effective preprocessing techniques
- TF-IDF-based document retrieval
- Context-aware response generation using LLM

The system performs well for structured queries such as:

- “When is the exam?”
- “Show syllabus”
- “Assignment deadline”

However, accuracy may slightly decrease for highly ambiguous or complex queries.

Response Time

The chatbot provides responses within **1–2 seconds**, ensuring a real-time conversational experience. The response time includes:

- Text preprocessing
- Document retrieval
- AI model processing

The use of optimized libraries and APIs contributes to the system’s fast performance.

Relevance of Responses

The relevance of responses is significantly improved due to the use of **context injection**. Instead of returning generic answers, the chatbot provides responses based on actual academic documents, making the output more reliable and informative.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

The AI-Based Academic Support Chatbot successfully addresses the limitations of traditional academic support systems by providing an intelligent, automated, and efficient solution for handling student queries. The system leverages **Natural Language Processing (NLP)** and **Machine Learning (ML)** techniques to understand user input and generate relevant responses in real time.

Through the implementation of preprocessing techniques such as tokenization, stopword removal, and lemmatization, along with feature extraction using TF-IDF, the chatbot is able to process and interpret natural language effectively. The integration of a Generative AI model further enhances the system's ability to provide context-aware and human-like responses.

The chatbot demonstrates high accuracy and efficiency in handling commonly asked academic queries related to exams, syllabus, and assignments. It significantly reduces the dependency on manual support systems, thereby minimizing workload on faculty and administrative staff. Additionally, the use of a web-based interface ensures ease of access and user-friendly interaction

The system architecture, which includes modular design, document retrieval, and logging mechanisms, ensures scalability, maintainability, and efficient performance. The results obtained during testing indicate that the chatbot is capable of delivering fast, accurate, and consistent responses.
In conclusion, the project successfully demonstrates how artificial intelligence can be applied to improve academic support systems, making them more efficient, accessible, and user-centric.

6.2 Limitations

Despite its effectiveness, the current implementation of the chatbot has certain limitations:

- The system has limited ability to understand highly complex or multi-intent queries
- It does not maintain long-term conversational context
- The accuracy of responses depends on the quality and coverage of the dataset and documents
- The chatbot primarily supports text-based interaction only
- Limited handling of ambiguous or vague user inputs

These limitations highlight areas where the system can be further improved.

6.3 Future Scope

The AI-Based Academic Support Chatbot has significant potential for future enhancements and expansion. Some of the possible improvements include:

- Integration with Real-Time Databases:
  Connecting the chatbot with institutional databases to provide dynamic and up-to-date information
- Advanced AI Models:
  Incorporating more advanced language models to improve understanding of complex queries and context
- Voice-Based Interaction:
  Enabling speech recognition and voice responses to enhance accessibility
- Multilingual Support:
  Supporting multiple languages to cater to a diverse user base
- Context-Aware Conversations:
  Implementing memory-based systems to maintain conversation history and improve interaction quality
- Mobile Application Integration:
  Deploying the chatbot as a mobile application for wider accessibility
- Personalized Responses:
  Providing customized responses based on user profiles and preferences
- Improved Analytics:
  Enhancing the analytics dashboard to provide deeper insights into user behaviour and system performance

These future enhancements can transform the chatbot into a fully intelligent academic assistant capable of handling complex interactions and providing personalized support.

Chapter 7: References

The following technical documentation, APIs, and software libraries were directly referenced and/or utilized during the research and development of the AI-Based Academic Support Chatbot:

Large Language Models (LLMs) & Compute Platforms

1. **Google Gemini API**: Initially researched and integrated for foundational generative processing and natural language understanding. Due to strict Free-Tier rate limits hindering rapid development, it was successfully substituted, but its prompt-engineering structure heavily influenced the final design.
    - *Google DeepMind. (2024). Gemini: A Family of Highly Capable Multimodal Models. Retrieved from https://deepmind.google/technologies/gemini/*
2. **Groq LPU Compute Engine**: Utilized as the primary API host to bypass traditional GPU compute bottlenecks, achieving extreme inference speeds.
    - *Groq, Inc. (2024). The LPU Inference Engine. Retrieved from https://groq.com/*
3. **Meta Llama-3 (8-Billion Parameter Model)**: The core foundational model accessed via Groq, responsible for both JSON-based Intent Classification and context-constrained academic answer generation.
    - *Meta AI. (2024). Introducing Meta Llama 3: The most capable openly available LLM. Retrieved from https://llama.meta.com/*

Application Frameworks & NLP Libraries

4. **Streamlit**: Selected as the primary frontend controller and Web Framework due to its rapid prototyping capabilities in Python.
    - *Streamlit. (2024). The fastest way to build and share data apps. Retrieved from https://streamlit.io/*
5. **Scikit-Learn**: Utilized specifically for the TfidfVectorizer and cosine similarity algorithms to construct the localized Semantic Search Engine.
    - *Scikit-learn developers. (2024). Machine Learning in Python. Retrieved from https://scikit-learn.org/stable/*
6. **PyPDF2**: Implemented as the primary Data Ingestion tool to parse raw, unformatted text from collegiate .pdf documentation.
    - *Phaseit, Inc. (2023). PyPDF2 Documentation. Retrieved from https://pypdf2.readthedocs.io/*

Environment & Security

7. **Python-Dotenv**: Referenced for securely abstracting API credentials via hidden .env environments to prevent security leakage.
    - *Python-dotenv standard library. Retrieved from https://pypi.org/project/python-dotenv/*

Knowledge Base:

8. Kes Shroff College: Referenced data for giving the knowledge base to bot for academic purposes https://kessc.edu.in/